`scripts/pipeline-build-test-distribute.sh` — User Guide

Last verified: 2026-08-25 (T046/T057 full-sequence wiring — `changelog_entry`, `distribute`, `docs_refresh`; independent audit of the `gate` mode, the evidence-directory derivation and `--help`)
Inheritance: HelixConstitution §11.4.18 (script documentation mandate) + Lava's Anti-Bluff Pact (Sixth/Seventh Laws) + §6.T.2 (Resource Limits)

Overview

The top-level orchestrator for this project's local build-test-distribute pipeline (`specs/002-build-test-distribute-pipeline/`). It sequences the pipeline's phase scripts against one shared run identifier, and consolidates their results into a single Pipeline Run Report per invocation.

What is wired today — and what deliberately is not

This section is the authoritative statement of what this script can currently do. It is deliberately explicit, because a pipeline that quietly does less than its name implies is the exact failure this feature exists to prevent.

Phase	Script	Wired?
`precondition`	`scripts/pipeline/phase-00-precondition.sh`	
`build`	`scripts/pipeline/phase-01-build.sh`	
`test`	`scripts/pipeline/phase-02-test.sh`	
`install_boot`	`scripts/pipeline/phase-03-install-boot.sh`	
`live_verify`	`phase-04-live-verify-api.sh` **and** `phase-04-live-verify-api-app.sh`	both halves
`changelog_entry`		(T046)

Phase	Script	Wired?
	`scripts/pipeline/phase-05a-changelog-entry.sh`	
`distribute`	`scripts/pipeline/phase-05-distribute.sh`	(T046) — **gate only; it cannot distribute**
`docs_refresh`	`scripts/pipeline/phase-06-docs.sh`	(T046)
`closure`	`phase-07-closure.sh`	the script does not exist — blocked on **T054 alone**

`closure` is **not** unimplemented-by-accident. **CORRECTED 2026-08-26 — this paragraph used to name T048/T049 as a co-blocker, and that is no longer true.** T048 and T049 (the carve-out in the Decoupled Reusable Architecture rule's "submodule fetch/pull is an EXPLICIT operator action, never automatic") both **landed 2026-08-23** under explicit operator approval, as did T040/T041; root `CLAUDE.md` now carries the `Automated Pipeline Pin-Advance Path` subsection with conditions (A)-(F). **No constitutional amendment blocks this phase.** The sole remaining blocker is **T054**, the dedicated review gate for `scripts/advance-all-submodules.sh` — the highest-blast-radius component in this feature, which has run four review rounds, returned APPROVE-WITH-FIXES every time, and has never returned a clean approval. T055 (the phase script itself) depends on it. `--until closure` is a usage error (exit `2`), never a silent no-op.

One stale citation of the old blocker deliberately remains in the script itself, at the `--until` error message on line 375: it is operator-facing output rather than a comment, so the 2026-08-26 comment-only correction pass left it for the commit that wires `closure`.

Wiring `distribute` did not make this script able to distribute. `phase-05-distribute.sh` is the §6.AA clause 8 refusal gate and today it is *only* that: it never invokes `scripts/firebase-distribute.sh`, writes no Distribution Record, and mutates `report.json` in no way. See "The distribute phase is a refusal gate" below for what its exit codes mean and how they are recorded.

R-004 ordering

`changelog_entry` runs **before** `distribute`, and the broader `docs_refresh` runs **after** it.

`scripts/firebase-distribute.sh` treats the `CHANGELOG.md` entry (its Gate 2) and the per-version `.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>.md` snapshot (its Gate 3) as

pre-existing inputs it verifies, never as things it authors. `spec.md`'s User Story 3 groups "distribute" and "refresh documentation" together, but the dependency direction for the CHANGELOG specifically is inverted from that naive reading. The broader documentation refresh has no such inversion and goes last.

The distribute phase is a refusal gate, and its exit 3 is neither pass nor fail

`phase-05-distribute.sh` defines exactly three exit codes:

Code	Meaning
`0`	A distribution completed. **Reserved, and unreachable today** — no distribute step is implemented.
`2`	**GATE REFUSED** (one or more of FR-009 / §6.AA clause 8 (A)–(H) failed, or a usage error). Refusal is the default.
`3`	**GATE QUALIFIED**, and the distribute step is not implemented.

Exit `3` is the interesting one, because the run report's `phases[].result` enum is `PASS | FAIL | SKIPPED` and **none of the three means what it means**:

- `PASS` would put a `distribute: PASS` entry into the report for a run that distributed nothing. SC-008 tells an auditor to read `report.json` *first*; a reader of that entry would conclude a distribution happened. Manufacturing the appearance of a distribution that did not occur is the §6.Z / §6.AK bluff class, one level up.
- `FAIL` would say the gate failed. It did not — it qualified.
- `SKIPPED` is the closest word, but `finalize_run_report` treats a SKIPPED phase as **not-PASS** (deliberately, and documented as such in its own docblock), so recording it would finalize *every* otherwise-perfect run to `outcome: FAIL` and exit non-zero. A pipeline whose exit code is `1` for a good run and `1` for a broken one has no exit code at all, and an always-red gate is ignored within a week — which is how a real failure then ships unnoticed.

So on exit `3` the orchestrator records no `phases[]` entry for `distribute` at all, does not halt, and discloses the fact on stdout and in the run-summary box:

```
  distribute:  GATE QUALIFIED — NOTHING WAS DISTRIBUTED (no
distribute step is
               implemented; report.json has no 'distribute' phase
entry and its
               'distributions' array is empty)
```

The absence of the entry is the honest record — nothing was distributed, so there is no distribution result to report — and the empty `distributions` array is its machine-readable half. This is the only path in the script that runs a phase and records nothing for it, and it is reachable only for a `gate`-mode phase exiting exactly `3`.

Every other exit code from that gate still fails the run and is recorded as `distribute: FAIL` by the orchestrator (the gate appends nothing of its own, so if the orchestrator does not record it, nobody does and `finalize_run_report`'s all-PASS rule is satisfied vacuously by the phases that *did* report). That includes exit `2` — the gate's own default refusal — and any code the gate never defined: an unmodelled state is read as failure, never as consent.

And a failure anywhere in a gate phase outranks a script that qualified. A phase may name several scripts. `distribute` names one today, but the registry supports more and `live_verify` already uses that shape. When an earlier script exits `3` and a later one fails, the phase **failed**: it is recorded `FAIL` and the run halts. The qualified-no-op path is reachable *only* when every script in the phase returned without failing.

> **Fixed 2026-08-25 (independent audit).** The orchestrator used to test the qualified-no-op condition *before* the phase's failure, which made the failure branch unreachable the moment any script in the phase had exited `3`. Such a run recorded nothing for the phase, and `report.json` — the artifact SC-008 sends an auditor to *first* — finalized to `outcome: "PASS"` while the process exit code said `1`, with the run-summary box printing `outcome: PASS` and `halted at: distribute` side by side. The condition order is now failure-first. Regression coverage: `tests/pipeline/test_orchestrator_gate_and_registry_audit.sh` CASE A, with CASE B guarding the opposite over-correction (an all-qualified multi-script gate must still record no entry and pass).

The default run now writes to the repository

`changelog_entry` writes a `CHANGELOG.md` entry and a per-version snapshot file; `docs_refresh` applies `research.md` R-002's stale-documentation fixes and regenerates the `.html`/`.pdf` siblings of every `.md` it changed. Both are what FR-013 / SC-006 ("zero manual documentation follow-up required") ask for, and both are now inside the default `--until docs_refresh`.

Two consequences worth stating plainly rather than discovering:

- A completed default run leaves the working tree **dirty on purpose**. The phase that commits those changes is `closure`, which is not wired. Until it is, committing is a human act.

- Because FR-000 requires a clean tree, a **second** run started before those changes are committed or discarded will refuse at `precondition`. That refusal is correct, not a bug.

`--until live_verify` reproduces the pre-T046 default exactly, and `--skip changelog_entry,distribute,docs_refresh` does the same for a run that should touch nothing.

Because an operator who types no flags at all never reads this guide or `--help`, a run that will genuinely reach one of those two phases **also announces it on stdout** before the first phase starts:

```
pipeline-build-test-distribute: NOTE — this run WRITES TO THE
REPOSITORY. Phase(s)
  changelog_entry docs_refresh edit tracked files (CHANGELOG.md,
its per-version
  snapshot, documentation and its regenerated .html/.pdf
siblings).
```

The notice is computed from the phases the run will actually reach: `--until` short of them, or `--skip` of them, suppresses it — a warning that is wrong teaches the reader to ignore the next one. Regression coverage: `tests/pipeline/test_orchestrator_gate_and_registry_audit.sh` CASE H (positive *and* both suppression cases).

`live_verify` covers both surfaces (since T037 landed, 2026-08-21): the running `lava-api-go` service, and the `:api-app` debug APK installed onto a Containers-submodule emulator per §6.AH driving its real boot-and-serve Challenge. A phase may own more than one script; they run in listed order and the phase halts at the first failure, so a later script can never append a PASS entry for a phase whose earlier half already failed. Both append their own `live_verify` entry to `phases[]` — the report schema permits that (`phases[]` declares no `uniqueItems`), and it is the honest shape, since `finalize_run_report` requires *every* entry to be PASS.

The api-app half establishes §6.AH compliance as a **checked fact**, not a logged claim: a poller samples `podman ps --filter name=lava-emu` throughout the run, and its provenance record is FAIL if no running emulator container was ever observed. It also requires Gradle's own JUnit XML and the Containers attestation row to agree, treating disagreement as a failure in itself.

Remaining honest caveat: the api-app half runs one AVD (API 34 phone) — the only emulator image cached on this host. That is live-verification, not the §6.AE.2 release-gate matrix.

Usage

```
./scripts/pipeline-build-test-distribute.sh [options] [repo-path]
```

Option	Meaning
`--until <phase>`	Stop after `<phase>` completes successfully. One of `precondition`, `build`, `test`, `install_boot`, `live_verify`, `changelog_entry`, `distribute`, `docs_refresh`. Default `docs_refresh` (the furthest wired phase). This is what makes `quickstart.md`'s per-user-story slices runnable: `--until test` is the US1 slice, `--until live_verify` is US1+US2, `--until docs_refresh` is US1+US2+US3.
`--skip <phase>[,<phase>…]`	Do not run the named phase(s). `precondition` may **not** be skipped.
`-h, --help`	Print the **whole** usage block (the script's header comment, ~190 lines) and exit `0`. It is produced by an awk scan that stops at the first non-comment line, not by a hardcoded line window — a fixed window silently truncates the moment the header is edited.

Naming a phase that is not wired (today that is only `--until closure`) is a **usage error (exit 2)**, not a silent no-op.

`precondition` cannot be skipped because it is the FR-000 safety boundary — a pipeline that can be talked out of checking its own preconditions has no safety boundary at all.

The optional positional `repo-path` is forwarded to every phase script as its own `[repo-path]` argument. It controls which repository the phases *inspect*; it does **not** relocate the Pipeline Run Report, which is always written relative to the current working directory (per `scripts/pipeline/lib/run-report.sh`'s documented contract). Run this script from the repo root, as with `scripts/ci.sh` and `scripts/tag.sh`.

Preconditions (FR-000)

Refuses to proceed (exit `2`) unless the working tree is on the `master` branch with a completely clean `git status`. This is the pipeline's sole safety boundary against an accidental unattended run from unreviewed or uncommitted work.

Required `.gitignore` entry. `.lava-ci-evidence/pipeline-runs/` MUST be gitignored (it is — `.gitignore` line 59, task T003). The run report is initialized *before* the precondition check, so that a

refusal to start is itself recorded as a run outcome. That means the run's own evidence directory exists on disk by the time FR-000's clean-tree rule is evaluated. Without the ignore rule the pipeline dirties the very tree it is about to test and can then never start. This script detects that specific situation and prints an explicit `DIAGNOSIS` naming the offending paths rather than leaving you to hunt for it.

Evidence and the run report

Every invocation creates a fresh, timestamped directory at `.lava-ci-evidence/pipeline-runs/<UTC-run-id>/` containing a `report.json` (schema: `specs/002-build-test-distribute-pipeline/contracts/pipeline-run-report.schema.json`), one `phase-NN/` subdirectory per phase script, and the phase's own Evidence Records. The subdirectory name carries the script's number **including any suffix letter**: `phase-05a-changelog-entry.sh` owns `phase-05a/` and `phase-05-distribute.sh` owns `phase-05/`, and they are deliberately not the same directory. Per `research.md` R-010, a later invocation never reads a prior run's directory as input — every run restarts fully from scratch (FR-018).

On **every** exit path once the report exists — success, phase failure, or interrupt — this script runs `append_interrupted_phase_if_any`, then `recompute_evidence_summary`, then `finalize_run_report`, **in that order**. The interrupted-phase step runs first so the outcome rule sees the FAIL phase it records.

The recompute step is not bookkeeping; it is load-bearing. `data-model.md`'s Validation rule makes `outcome: "PASS"` conditional on `evidence_summary.rejected_by_anti_bluff == 0`, and `finalize_run_report` implements that rule literally — but the counter it reads is seeded to `0` by `init_run_report` and is only ever populated by `recompute_evidence_summary` scanning the real Evidence Records on disk. Skipping the recompute silently turns the anti-bluff half of the outcome rule into a no-op. Regression coverage: `tests/pipeline/test_run_report_evidence_summary.sh` CASE 3.

Consequently, a run whose phases all exited `0` but whose finalized `outcome` is `FAIL` (because an Evidence Record was REJECTED by anti-bluff validation) **exits non-zero**. The finalized outcome is authoritative over the individual phase exit codes.

An interrupted run can never finalize to PASS

Added 2026-08-23, from the first genuine end-to-end runs of this pipeline.

The defect. Run `2026-08-23T10-15-30Z` was killed mid-build. The process exit code was honest — `1`, `halted at: build`. The `report.json`, which SC-008 tells an auditor to read *first*, said:

```
"outcome": "PASS"
"phases":  [{"name":"precondition","result":"PASS"}]
"build_artifacts": []        "evidence_summary": {"total": 0, ...}
```

A run that never got past its first phase, produced zero artifacts and zero Evidence Records, reported success.

`finalize_run_report` could not have caught it. Its rule is `(.phases | length) > 0 and (.phases | all(.result == "PASS"))`. The `length > 0` guard closes the **empty** case; it cannot close the **truncated prefix** case, because a truncated run's `phases[]` is a perfectly valid all-PASS list — merely *shorter* than the run was supposed to be. Nothing in the report distinguished "ran everything and passed" from "stopped after phase 1". That is the same vacuous-pass class `phase-02-test.sh`'s PASS condition 4 closed (a phase passing on zero Evidence Records), sitting one level up.

The fix. A per-**script** in-flight marker, via three helpers in `scripts/pipeline/lib/run-report.sh`:

Helper	Called	Effect
`mark_phase_in_flight <run_id> <phase>`	immediately **before** a phase script is invoked	writes the phase name to `<run_dir>/.phase-in-flight`
`clear_phase_in_flight <run_id>`	the instant that script **returns**	removes the marker; removing an absent marker is not an error
`append_interrupted_phase_if_any <run_id>`	on **every** close path, before recompute/ finalize	a surviving marker becomes an honest `<phase>: FAIL` entry, then the marker is cleared so the operation is idempotent

Three design points worth keeping:

- **A file, not a shell variable.** The orchestrator can be killed outright; a variable dies with the process, a file survives so a later reader can still tell the run was interrupted. It lives inside the run directory, so it is per-run by construction and cannot

leak between runs — and that directory is gitignored, so the marker never dirties the working tree.
- **Per script, not per phase.** The marker means "*a phase script is executing right now*", not "*a phase started*". A script that returns was not interrupted, whatever its exit code, and has already recorded its own result. Marking per phase appends a phantom second FAIL to an ordinary phase failure, claiming the phase was interrupted when it ran to completion and failed. This was caught and corrected during the fix itself.
- **The report schema was not widened.** `pipeline-run-report.schema.json` is `additionalProperties: false`, and an interrupted phase genuinely did not pass — so this is expressed in the existing `phases[]` field. Recording it as FAIL states a fact the report was missing rather than inventing a field.

When a marker is found, the orchestrator prints to stderr:

```
pipeline-build-test-distribute: run was INTERRUPTED during phase '<name>' — recorded as FAIL. A run that did not finish is not a run that passed.
```

Regression coverage. `tests/pipeline/test_interrupted_run_never_passes.sh` — 3 cases, 6 checks. The RED run reproduced the real defect verbatim (`outcome is 'PASS', expected FAIL`); cases 2 and 3 were **green before** the fix, so a blanket fail-everything change could not have satisfied the suite. Case 3 asserts the marker is per-run and never leaks between runs. Verified additionally by real orchestrator runs against a disposable fixture: clean → exit 0, `outcome: PASS`, exactly one phase entry with no phantom FAIL; dirty → exit 2, `outcome: FAIL`, exactly one `precondition` entry with no duplicate; marker file absent after a completed run.

Each phase script gets its own evidence directory

The per-phase evidence directory is derived from the script's **own** number, including any letter suffix, so it matches the directory that script writes into: `phase-05a-changelog-entry.sh` owns `<run>/phase-05a` and `phase-05-distribute.sh` owns `<run>/phase-05`, and they are **not** the same directory. A fixed-width slice of the script name returns `05` for both and silently merges their evidence.

> **Audited 2026-08-25.** The suffix-aware derivation was correct, but nothing tested it: reverting that one line to the pre-fix fixed-width form left 117 checks green across five orchestrator suites, none of which ever looked at a directory name. `tests/pipeline/test_orchestrator_gate_and_registry_audit.sh` CASE D now

> asserts that a full run produces exactly eight distinct evidence directories including both `phase-05` and `phase-05a`, and that the derivation is not a fixed-width slice.

Exit codes

Code	Meaning
`0`	Every phase that ran reported PASS, and the finalized `outcome` is `PASS`. A `distribute` gate that QUALIFIED without distributing anything (its exit `3`) does not change this, and does not claim a distribution happened.
`1`	A phase genuinely failed, **or** the finalized `outcome` is `FAIL` (including the all-phases-passed-but-evidence-rejected case).
`2`	Usage error, or the FR-000 precondition check failed (propagated verbatim from `phase-00-precondition.sh`, never hardcoded).

Individual phase scripts

Per FR-005, every phase is independently invocable outside this orchestrator. All phase scripts except `phase-00` share the signature `<run_id> [repo-path]`. See `specs/002-build-test-distribute-pipeline/contracts/cli-contract.md`.

Who records a phase's `phases[]` entry is declared per phase in the orchestrator's `PHASES` registry, as its third field:

Mode	Meaning	Phases
`yes`	The phase script calls `append_phase_result` itself; the orchestrator must not append a second entry (a duplicate would corrupt the all-PASS outcome rule). The contract is still **verified**, not trusted, when the phase fails — see `phase_nonpass_count`.	`build`, `test`, `install_boot`, `live_verify`, `changelog_entry`, `docs_refresh`
`no`	The phase script does not append; the orchestrator appends PASS on exit `0` and FAIL otherwise.	`precondition`
`gate`	The phase script does not append **and** has a third outcome that is neither pass nor fail. Exit `0` → PASS; exit `3` → *no entry*; anything else → FAIL and halt.	`distribute`

Phase names are constrained to the `phases[].name` enum in `contracts/pipeline-run-report.schema.json`. The schema is the closed set; the registry does not get to invent names.

Verification record

The phase wiring was verified by real execution against disposable fixture git repositories (never against real `master`):

- Clean `master` fixture, `--until precondition` → exit `0`, `outcome: PASS`, exactly one `phases[]` entry.
- Fixture dirtied with a real tracked-file change → exit `2`, `outcome: FAIL`, halted at `precondition`, no false-positive diagnosis.
- Fixture with the `.gitignore` rule removed → exit `2` **plus** the `DIAGNOSIS` block naming the untracked run-directory paths.
- All usage guards (`--help`, unknown `--until`, not-yet-wired `--until closure`, `--skip precondition`, unknown option) → exit as specified.

The full end-to-end run across all eight wired phases is task **T062**, a review gate that must first be executed on a disposable branch.

T046 / T057 (2026-08-25). The full-sequence wiring is covered by `tests/pipeline/test_pipeline_full_sequence_wiring.sh`, which substitutes synthetic phase scripts for the real ones (nothing there invokes Gradle, systemd, podman, an emulator or Firebase) and asserts on order, on `report.json`, and on the exit code. It was built RED-then-GREEN: 30 failing checks before the wiring landed, all passing after. Four deliberate mutations were rehearsed against it and each was caught:

Mutation	Observed failure
map gate exit `3` → `distribute: PASS`	`FAIL: report.json contains a distribute entry (…, ('distribute', 'PASS'), …) for a run that distributed nothing`
map gate exit `3` → `distribute: SKIPPED`	`FAIL: gate exit 3 made the whole run exit 1 — every otherwise-good run would fail` (+ 4 more)
treat *every* non-zero gate code as the no-op (so the gate's failure cannot fail the run)	`FAIL: a REFUSED gate still exited 0 — a phase whose failure cannot fail the run is not wired, it is decorative` (+ 4 more)
invert the R-004 order (`docs_refresh` before `distribute`)	`FAIL: changelog_entry at 7, distribute at 0 — R-004 requires changelog first` (+ 7 more)

Independent audit, 2026-08-25 — `tests/pipeline/test_orchestrator_gate_and_registry_audit.sh` (48 checks). It was built RED-then-GREEN: 4 failing checks before the fixes landed, all passing after. Its own falsifiability rehearsals, each run against a temp-tree copy so the shipping script was never mutated:

Mutation	Observed failure
check `gate_no_op` before `phase_exit` again (the defect)	`FAIL: report.json says outcome='PASS' for a run that halted on a failure` (+ 2 more)
revert `phase_dir` to `${script_name:6:2}`	`FAIL: a full run produced 7 evidence directories, expected 8: phase-00 … phase-05 phase-06` (+ 2 more)
revert `_usage()` to a hardcoded line window that truncates to nothing	`FAIL: --help printed only 1 lines — the header is ~190; a fixed window or an early stop has truncated it` (+ 9 more)
remove the run-time repository-writing notice	`FAIL: a default run gives no run-time notice that it writes to the repository`

The same suite's CASE F drives each of the **eight** phases individually into a script that dies without appending, and asserts each one fails the run *and* is recorded `FAIL`. Before it, `install_boot` and the first half of `live_verify` had no such coverage anywhere. CASE G asserts that exit `3` is special only for a `gate` phase — from `build`, `live_verify`, `changelog_entry` or `docs_refresh` it is an ordinary failure, because the QUALIFIED meaning belongs to the gate contract, not to the number.

Updated 2026-08-23. The first genuine end-to-end runs of this pipeline have since been executed under tasks T031 / T058 / T061, and they are what surfaced the interrupted-run defect above and the `LAVA_STRESS_CHAOS_EVIDENCE_DIR` run-isolation defect documented in `docs/scripts/pipeline/phase-02-test-go.sh.md`. Recorded outcomes: T031 DONE (the US1 slice ran green with 1540 Evidence Records — 1528 PASS / 0 FAIL / 12 SKIPPED / 0 REJECTED); T058 DONE (run `10-15-30Z` killed, run `10-17-26Z` started fresh with zero cross-references to it); T061 DONE with a **negative** result — `scripts/ci.sh --changed-only` exits 1 on this branch on a §6.H credential-regex false positive against another stream's fixture file. UNCONFIRMED: whether the T062 review gate itself is satisfied by those runs is an operator determination, and this document does not claim it. See `specs/002-build-test-distribute-pipeline/progress.yml` (`t031_t058_t061_verdicts`) for the full verdicts.

Falsifiability

`scripts/pipeline/lib/anti-bluff-validate.sh`, `scripts/pipeline/lib/run-report.sh`'s evidence aggregator, and `scripts/pipeline/phase-00-precondition.sh` each carry a hermetic suite under `tests/pipeline/`, each built RED-then-GREEN. The evidence aggregator additionally carries a recorded mutation rehearsal: disabling its `REJECTED` detection makes CASE 3 fail with `outcome is 'PASS', expected FAIL`, and reverting restores the pass.

Maintenance

When this script is modified, update this document in the same commit (`CM-SCRIPT-DOCS-SYNC` requires it). Per §11.4.18, the documentation MUST stay in sync with the codebase.

Cross-references

- `specs/002-build-test-distribute-pipeline/spec.md`, `plan.md`, `research.md`, `data-model.md`, `contracts/`, `tasks.md` — the full design and task breakdown this script implements.
- `scripts/pipeline/lib/run-report.sh` — `mark_phase_in_flight` / `clear_phase_in_flight` / `append_interrupted_phase_if_any`.
- `tests/pipeline/test_interrupted_run_never_passes.sh` — the interrupted-run suite.
- `tests/pipeline/test_pipeline_full_sequence_wiring.sh` — the T046/T057 full-sequence wiring suite (order, the gate's tri-state exit contract, failure propagation).
- `tests/pipeline/test_orchestrator_gate_and_registry_audit.sh` — the independent audit suite (the `gate` mode's failure-first ordering, per-script evidence directories, `--help` content, per-phase failure propagation).
- `tests/pipeline/test_phase_05_distribute_gate.sh` — the distribute gate's own 24-case suite.
- `specs/002-build-test-distribute-pipeline/research.md` R-004 — the changelog-before-distribute ordering decision.
- `docs/scripts/pipeline/phase-02-test-go.sh.md` — the run-isolation seam that lets a run repeat.
- `docs/helix-constitution-gates.md` — gate inventory.
- HelixConstitution `Constitution.md` §11.4.18 (the mandate).